TF-IDF + Logistic Regression

IE4483 | Baseline method and implementation plan

Prepared for Person 1: Data and baseline | 8 November 2026

1. What are the two components?

TF-IDF is a feature representation: it turns text into numerical vectors. Logistic Regression is a classifier: it learns how to combine those features to predict sentiment. Despite its name, Logistic Regression is used here for classification, with 0 = negative and 1 = positive. [1, 2]

Review text -> tokenization -> sparse TF-IDF vector -> weighted linear score -> sigmoid -> predicted sentiment.

For this project, fit the entire pipeline on the saved training partition, then evaluate on the saved validation partition. The official test set is outside this development step.

Saved partition	Rows	Negative	Positive
Training	5,921	866	5,055
Validation	1,480	216	1,264

Why use it as our baseline?

This is a practical first experiment for our small dataset: it provides a reproducible reference before the GRU comparison. It needs no pretrained embeddings or GPU. Inspecting learned coefficients can help identify words associated with each class. Actual speed, memory and accuracy remain measurements to collect, rather than assumed results.

Input and output dimensions

Let V be the number of retained training features, including bigrams. The numerical training matrix has shape (5,921, V), validation has shape (1,480, V), and the binary classifier has one learned coefficient vector of length V plus an intercept. predict_proba returns two columns, ordered by the classifier's classes. [2]

V is not the raw JSON column count, and it is not yet known: it depends on training vocabulary and filtering. The full-pool EDA vocabulary count must not be reused as this fitted feature dimension.

Current status

This document describes the proposed baseline and prepared code. No real-data baseline training has been run for this work. Earlier README scores used a different split and tokenizer, so they are not current shared-split results.

2. How TF-IDF represents a review

Term frequency and inverse document frequency

A feature can be a word or a word pair. Let count(t,d) be its occurrences in review d; df(t) is the number of training reviews containing it, and N is the training-review count. The prepared configuration uses sublinear term frequency and smoothed IDF. [1, 3]

TF(t,d) = 1 + ln(count(t,d)) when count > 0; otherwise 0.

IDF(t) = ln((1 + N) / (1 + df(t))) + 1.

u(t,d) = TF(t,d) * IDF(t).

x(d) = u(d) / sqrt(sum over features of u(t,d)^2).

Repeated terms grow more slowly under log scaling. IDF downweights terms appearing in many training documents. L2 normalization puts non-empty review vectors on the same length scale; it is not L2 regularization of classifier weights. [1, 3]

Small numerical example: illustrative, not our dataset

Use three miniature training documents: 'comfortable shoes', 'not comfortable', and 'poor fit'. For clarity, use unigrams and min_df=1 here; the real configuration uses min_df=2 and bigrams too.

Feature	Document frequency	Smoothed IDF
comfortable	2 of 3	ln(4/3) + 1 = 1.288
shoes	1 of 3	ln(4/2) + 1 = 1.693
not	1 of 3	ln(4/2) + 1 = 1.693

The first document has normalized values approximately comfortable = 0.605 and shoes = 0.796; all other entries are zero. This worked example is calculated from the formulas above.

Word pairs and sparse storage

With ngram_range=(1,2), 'not comfortable' contributes the two unigram features and the bigram 'not comfortable', if retained in training. Bigrams capture local order, but not full sentence structure. Words outside the fitted vocabulary contribute no feature. [1, 3]

Most vocabulary features do not occur in a given review, so the representation is sparse. Store nonzero values and their indices instead of allocating an N-by-V dense array. A large V alone does not determine memory consumption: the number of nonzeros also matters. [3]

3. How Logistic Regression predicts sentiment

From feature weights to probability

For TF-IDF vector x, the model computes a linear score z and maps it through the sigmoid. A coefficient above zero increases the positive-class score when that feature is present; a negative coefficient decreases it. Learned associations are not causal explanations. [2, 4]

z = w dot x + b.

p(y = 1 | x) = 1 / (1 + exp(-z)).

Predicted class = 1 when p > 0.5; otherwise 0.

The threshold follows the prepared binary prediction rule. A probability is the model's estimate, not a guarantee of correctness. Class weighting changes the learning objective, so weighted outputs need not match the observed class prevalence without calibration.

Illustrative scoring example

Suppose, purely for illustration, that the unigram weights are comfortable = +1, not = -2, shoes = 0, with intercept 0. Using the preceding toy vectors, 'comfortable shoes' has z about 0.605 and positive probability about 0.647. 'not comfortable' has z about -0.987 and probability about 0.272. These are hypothetical weights, not learned project results.

What is learned during training?

Training adjusts coefficients and intercept to reduce binary log loss, with an L2 penalty discouraging excessively large coefficients. A confident wrong prediction incurs a large log loss. C controls inverse regularization strength: smaller C applies stronger regularization. The prepared initial value is C=4, not a proven optimum. [2, 4]

Per-example log loss = -[y * ln(p) + (1-y) * ln(1-p)].

Handling the minority negative class

The training partition has 866 negative and 5,055 positive examples. With class_weight='balanced', class k receives weight N / (2 * n_k), calculated from training labels only. This gives negative weight 3.419 and positive weight 0.586: negative mistakes receive about 5.84 times as much weight. [2]

Weighting does not oversample, remove rows or change validation labels. Its usefulness must be tested: it may improve negative recall while reducing precision or overall accuracy. Compare weighted and unweighted models on the same validation data.

Limits worth investigating

Unigrams lose order; bigrams retain only short-range order. Sarcasm, long contrasts and unseen phrasing may remain difficult. Domain-specific vocabulary may transfer poorly to hotel reviews. These are hypotheses for error analysis, not evidence that the GRU will necessarily outperform this baseline.

4. Proposed experiment on the shared split

Initial configuration (not yet tuned)

Component	Setting	Reason for this starting choice
Text processing	Lowercase; retain contractions, digits and one-character words	Preserve negation and useful size/rating tokens
Cleaning	No stop-word removal, stemming or lemmatization	Avoid discarding sentiment-bearing words
Features	Word unigrams + bigrams; min_df=2	Include short phrases; exclude features appearing in one training review
Weighting	Sublinear TF; smoothed IDF; L2 normalization	Temper repetition and normalize vector magnitude
Classifier	Logistic Regression; lbfgs; L2 default; C=4	Reuse a starting value from the exploratory baseline
Imbalance	class_weight='balanced'	Test a response to minority-class errors
Optimization	max_iter=2000; random_state=42	Record settings; fail on non-convergence

The actual tokenizer is (?u)\b\w+(?:['']\w+)*\b. It retains 'don't' as one token, but is not the regex used for the earlier descriptive token counts. Lowercasing removes capitalization cues; punctuation outside contractions is not modeled. These choices are explicit ablations for later work.

Leakage controls

The script verifies source/output hashes and original row indices against split_manifest.json, complete source coverage and zero exact-text overlap. It then fits a single scikit-learn Pipeline using training reviews and labels only. Validation calls transform/predict, never fit. This follows the documented separation between training and evaluation transformations. [5]

Exact-text grouping protects identical copies; near-duplicates can still exist. Keep the existing split fixed during model comparison rather than repeatedly selecting favorable partitions.

Evaluation and model selection

Accuracy measures the fraction correct. Macro-F1 averages the two class F1 values equally; per-class precision and recall expose minority errors. A confusion matrix will use rows = true labels, columns = predictions, ordered [negative, positive]. [6]

On this validation set, always predicting positive gives 1,264/1,480 = 85.41% accuracy, zero negative recall and macro-F1 about 0.461. These are analytically calculated reference values, not trained-model performance. Select experiments primarily by macro-F1 and inspect negative recall.

Start with the proposed setting, then compare unigram versus unigram+bigram and weighted versus unweighted models; optionally test C in {0.5, 1, 4}. Validation guides these choices, so the selected validation score is not an independent final test estimate. No sweep is launched in this preparation step.

5. Code readiness and evidence to collect

Prepared entry point

Run from the repository root using the project environment:

.venv/bin/python baseline/train.py --dry-run

.venv/bin/python baseline/train.py

The first command checks the shared files and prints configuration without training or writing artifacts. The second performs training when ready. The implementation lives in baseline/train.py; baseline/README.md describes options. Focused tests use toy data to check vocabulary isolation, saved-model consistency and minority-class metric handling.

Outputs of a future training run

Artifact in artifacts/tfidf_logreg/	Purpose
model.joblib	Save vectorizer and classifier together for consistent inference
metrics.json	Scores, confusion matrix, timings, sparse matrix size, process peak memory, versions and fingerprints
validation_predictions.csv	Original row index, review, truth, prediction and positive probability for error analysis

Training time includes vectorizer fitting and classifier optimization; validation inference time includes text transformation and probability prediction. Sparse-array bytes exclude vocabulary/Python overhead. Process peak RSS includes imports and diagnostics; model-file size measures serialized storage. These quantities must not be treated as interchangeable memory measurements.

Connection to the project report

Section (a): describe text processing and numerical feature construction. Section (f): compare measured feature dimensions, accuracy/macro-F1, runtime and memory with the GRU on the same split and comparable hardware. Section (g): use later transfer experiments or reasoned domain-adaptation discussion; do not claim hotel-review accuracy without target labels.

References: official scikit-learn documentation

[1] TfidfVectorizer API:
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[2] LogisticRegression API:
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[3] Feature extraction guide, TF-IDF: https://scikit-learn.org/stable/modules/feature_extraction.html#tfidf-term-weighting

[4] Linear models guide, Logistic Regression:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

[5] Common pitfalls, data leakage: https://scikit-learn.org/stable/common_pitfalls.html#data-leakage

[6] Classification metrics: https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

Local evidence: data/train_splitted.json, data/valid_splitted.json and data/split_manifest.json. Theory checked against the official documentation on 8 November 2026. This is a methods/preparation report, not an experiment-results report.